Сжатие данных
(data compression)

Определение:

Сжатие данных — алгоритмическое (обычно обратимое) преобразование данных, производимое с целью уменьшения занимаемого ими объёма.

8 битов на пиксель
(256 цветов)

4 бита на пиксель
(16 цветов)

2 бита на пиксель
(4 цвета)

- Сжатие возможно только тогда, когда в данных есть какие-то закономерности (избыточность).

Классы сжатия

- **Сжатие без потерь** — это такое уменьшение объёма данных, при котором можно восстановить их исходный вид без искажений.

- **Сжатие с потерями** — это такое уменьшение объёма данных, при которых распакованный файл может отличаться от оригинала. Сжатие с потерями используют при кодировании фотографий, звука и видео.

Сжатие с потерями

Тип данных	Тип файла после сжатия	Степень сжатия
Графика	.JPG	до 99%
Видео	.MPG	
Звук	.MP3	

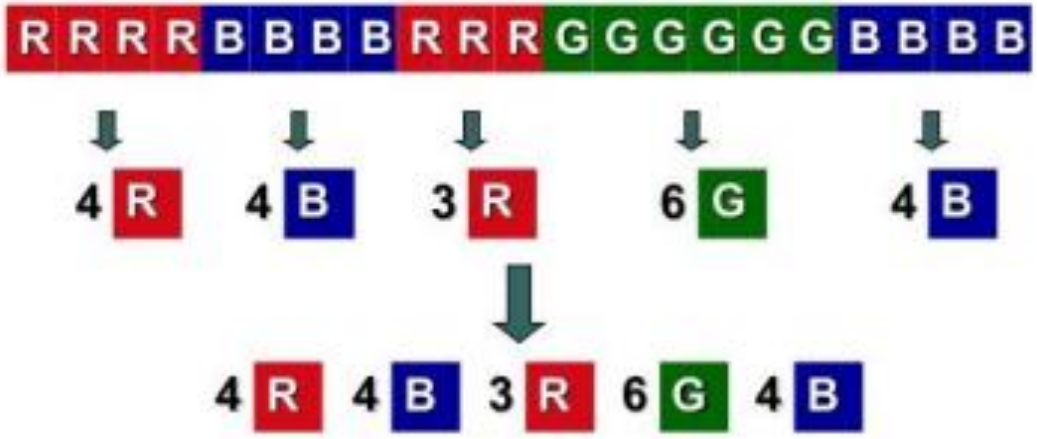
Сжатие без потерь

Тип данных	Тип файла после сжатия	Степень сжатия
Графика	.GIF .TIF .PCX	До 50%
Видео	.AVI	
Любой тип	.ZIP .ARJ .RAR .LZH	

Сжатие без потерь

Алгоритмы серии RLE
(run-length encoding или кодирование повторов)

Идея: повторяющиеся группы элементов заменяются на числа повторов.



Похоже на химию, когда мы вместо HHO пишем H2O

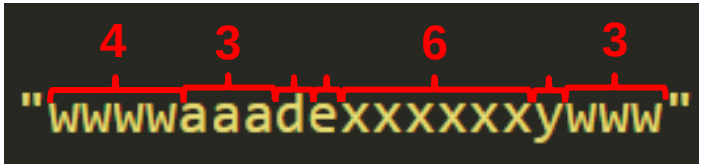
Код RLE алгоритма

```
void printRLE(string str)
{
    int n = str.length();
    for (int i = 0; i < n; i++) {
        // Count occurrences of current character
        int count = 1;
        while (i < n - 1 && str[i] == str[i + 1]) {
            count++;
            i++;
        }
        // Print character and its count
        cout << str[i] << count;
    }
}


//Driver code
int main()
{
    string str = "wwwwaaadexxxxxxywww";
    printRLE(str);
    return 0;
}
```

Время работы: **O(n)**

Дополнительная память: **O(1)**



w4a3d1e1x6y1w3

Арифметическое кодирование (Arithmetic coding)

— алгоритм сжатия информации без потерь, который при кодировании ставит в соответствие тексту вещественное число из отрезка [0;1).

- Данный метод, как и алгоритм Хаффмана, является энтропийным, то есть длина кода конкретного символа зависит от частоты встречаемости этого символа в тексте.

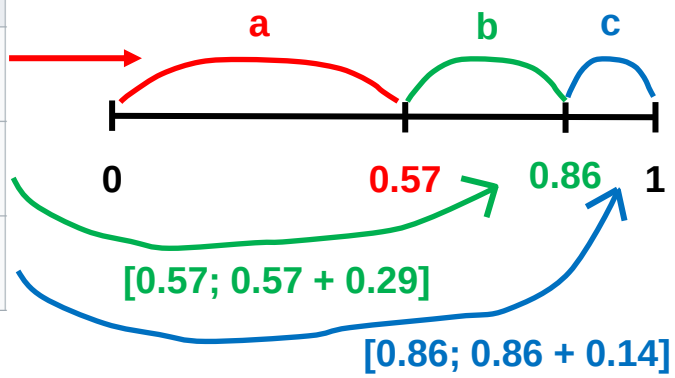
- Арифметическое кодирование показывает более высокие результаты сжатия, чем алгоритм Хаффмана, для данных с неравномерными распределениями вероятностей кодируемых символов.

Алгоритм кодирования:

На вход алгоритму передаются текст для кодирования и список частот встречаемости символов.

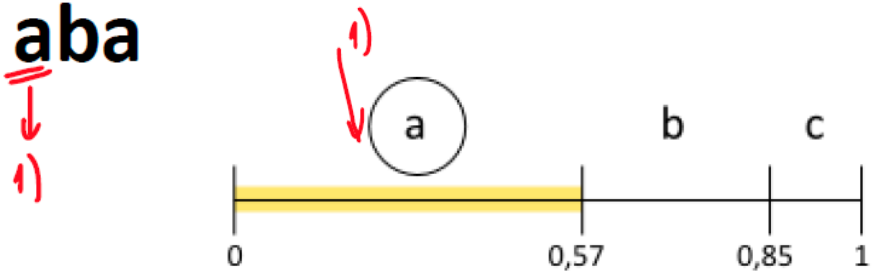
1. Рассмотрим отрезок [0; 1) на координатной прямой.
2. Поставим каждому символу текста в соответствие отрезок, длина которого равна частоте его появления.

Символ	Частота появления
a	0.571429
b	0.285714
c	0.142857



Алгоритм кодирования:

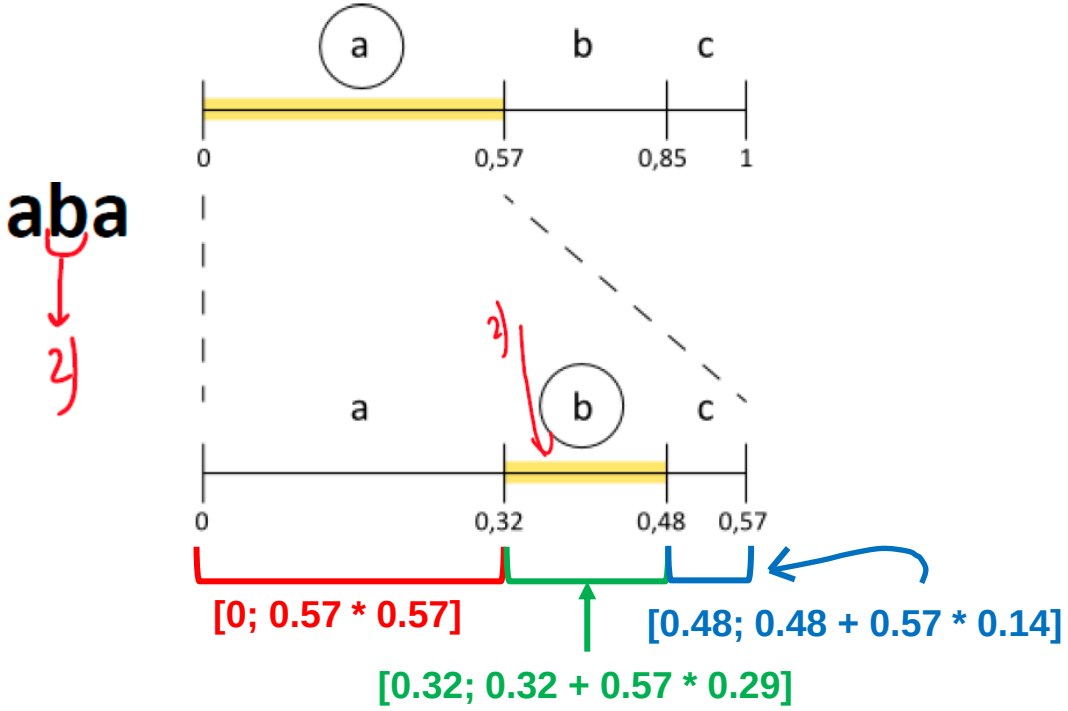
3. Считаем символ из входного потока и рассмотрим отрезок, соответствующий этому символу. Разделим этот отрезок на части, пропорциональные частотам встречаемости символов.



Теперь наш новый «полный» отрезок – это отрезок [0; 0.57]

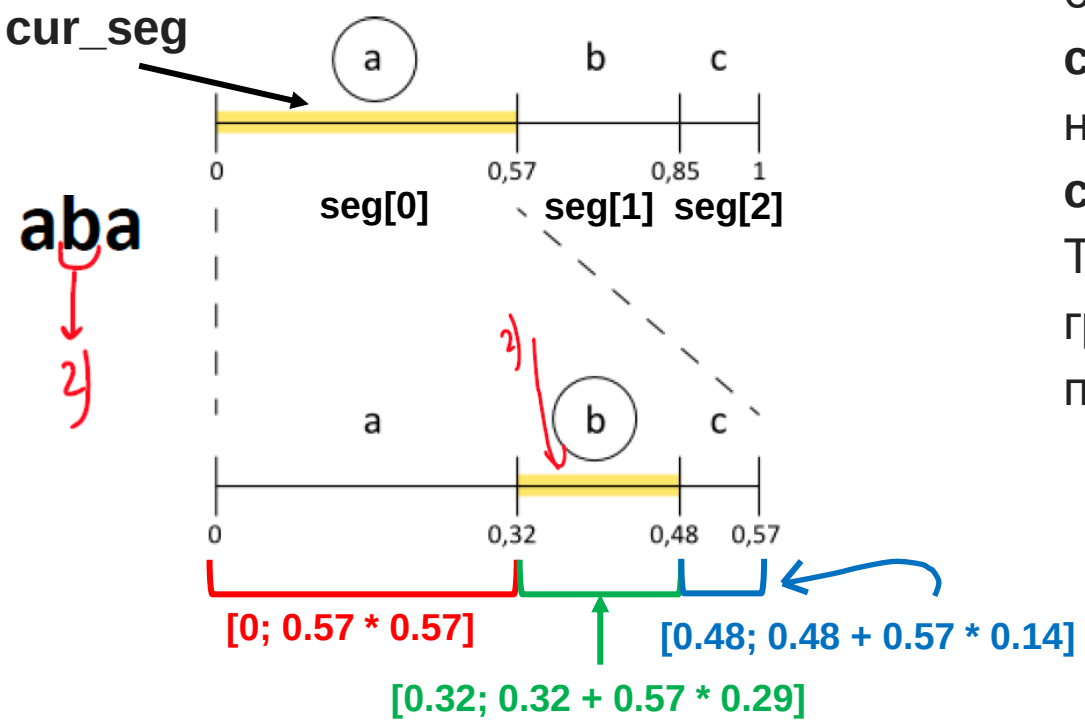
Алгоритм кодирования:

4. Повторим пункт (3) до конца входного потока.



Алгоритм кодирования:

В общем случае пересчитывается так:

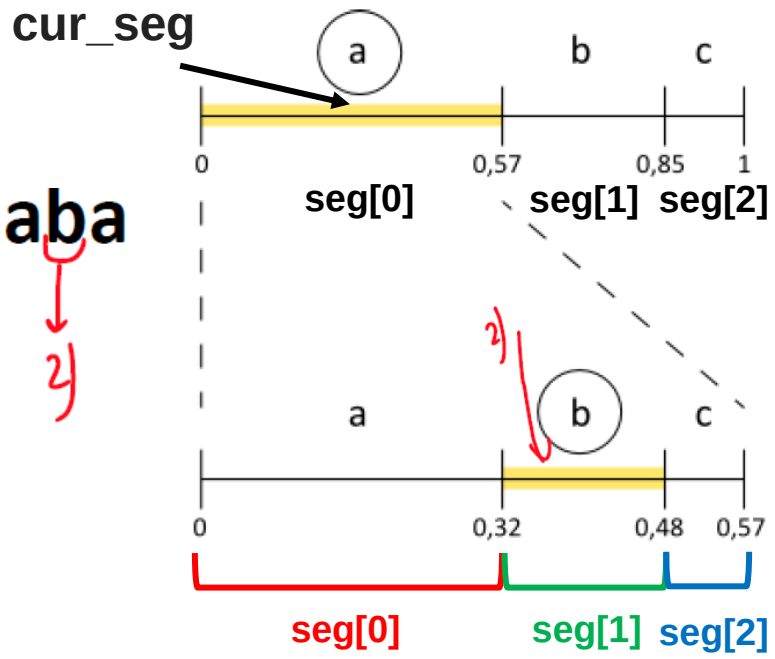


Допустим наша предыдущая буква попала в отрезок **cur_seg**. Тогда границами нашего нового отрезка будут **cur_seg.left** и **cur_seg.right** Тогда для новой буквы границы отрезка **i** пересчитываются как

[seg[i - 1].right ;
seg[i – 1].right
+ len(cur_seg) * prob(let[i])]

Алгоритм кодирования:

В общем случае пересчитывается так:



Будем считать, что отрезок **seg[-1]** имеет границы: **[cur_seg.left; cur_seg.left]**

[seg[i - 1].right ;
seg[i – 1].right
+ len(cur_seg) * prob(let[i])]

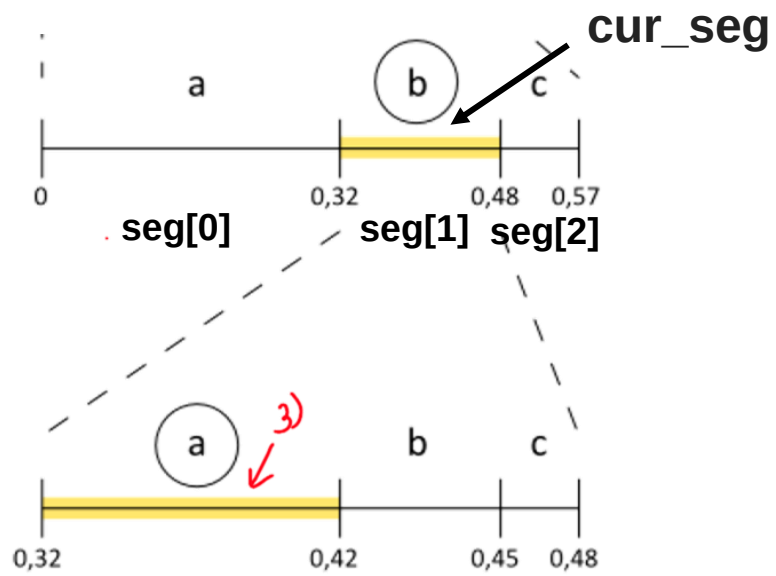
[seg[-1].right; seg[-1].right + len(cur_seg) * prob(let[0]])] =
= [0; 0 + 0.57 * 0.57]

[seg[0].right; seg[0].right + len(cur_seg) * prob(let[1])] =
= 0.32; 0.32 + 0.57 * 0.29]

[seg[1].right; seg[1].right + len(cur_seg) * prob(let[2])] =
= [0.48; 0.48 + 0.57 * 0.14]

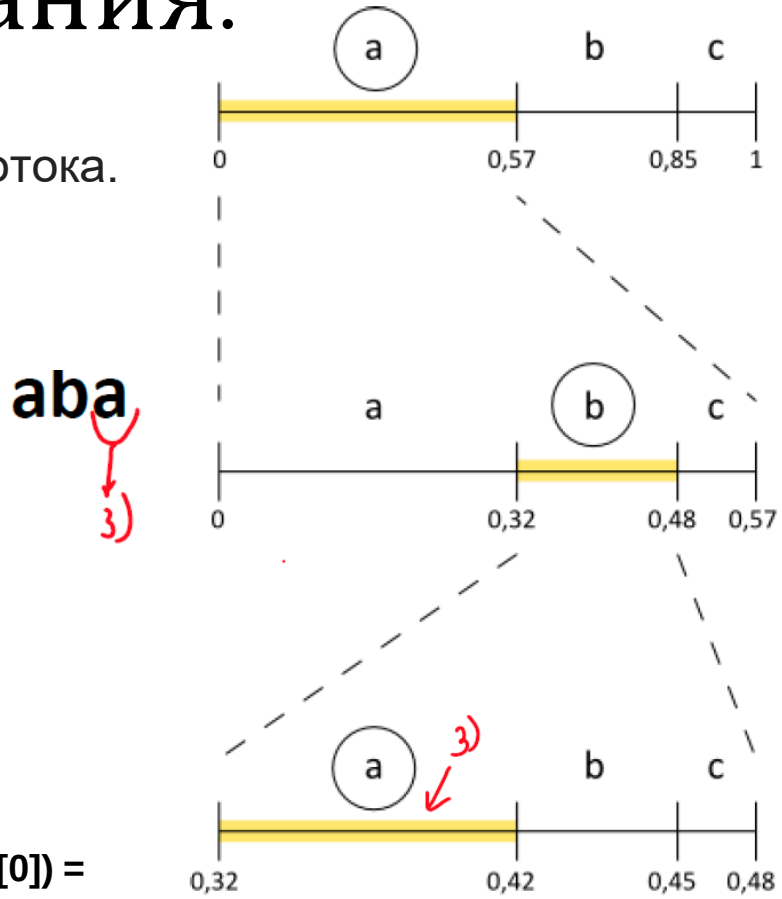
Алгоритм кодирования:

4. Повторим пункт (3) до конца входного потока.



seg[0].left =
= cur_seg.left

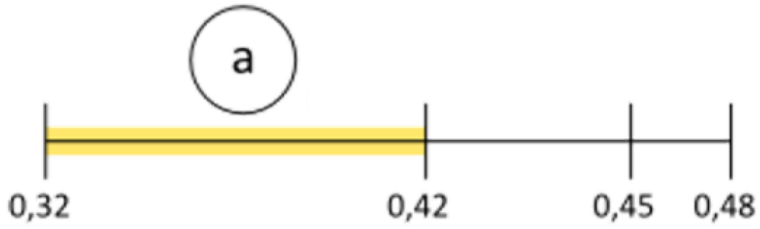
seg[0].right =
= seg[-1].right + len(cur_seg) + prob(let[0]) =
= 0.32 + 0.16 * 0.57 = 0.42



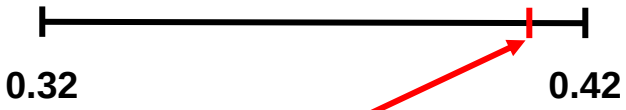
Алгоритм кодирования:

5. Выберем любое число из получившегося отрезка, которое и будет результатом арифметического кодирования.

Отрезок, получившийся после выполнения последнего шага



Выберем произвольное число



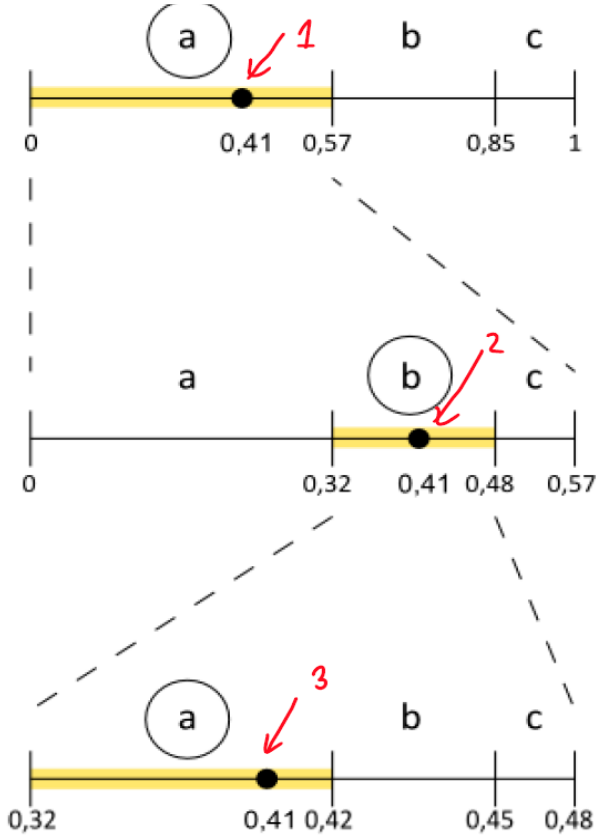
Допустим, 0.41

$$0.41_{10} \longrightarrow 2^5 + 2^3 + 2^0 = \underbrace{101001}_{6\ \text{бит}}{}_2$$

aba = 24 бита

8 8 8

Декодирование способ 1



Декодируем

0.41 в текст

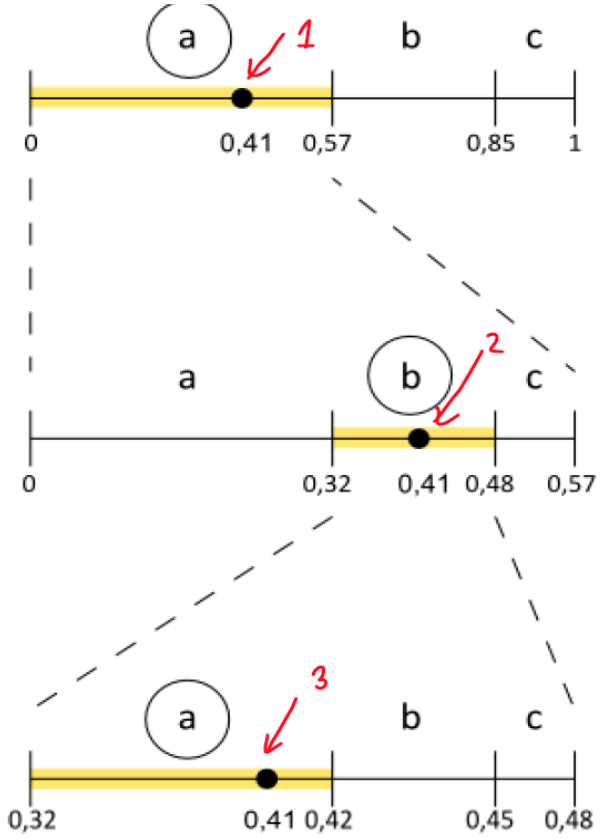
способ 1

Мы знаем

stop = 3 шага

aba

Декодирование способ 1



0.41 ∈ [0; 0.57] ⟶ **a**

Декодируем

0.41 в текст

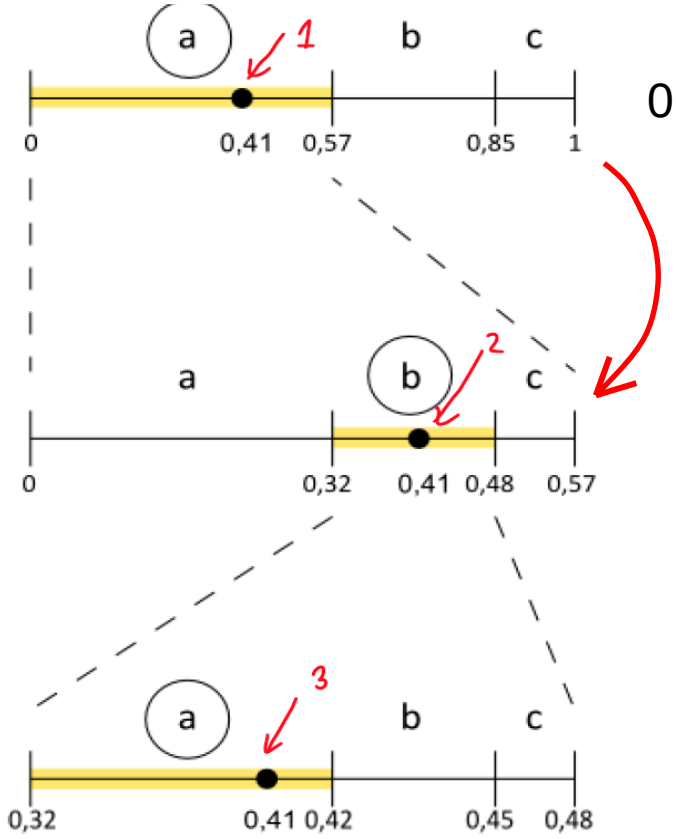
способ 1

Мы знаем

stop = 3 шага

aba

Декодирование способ 1



0.41 ∈ [0; 0.57] ⟶ **a**

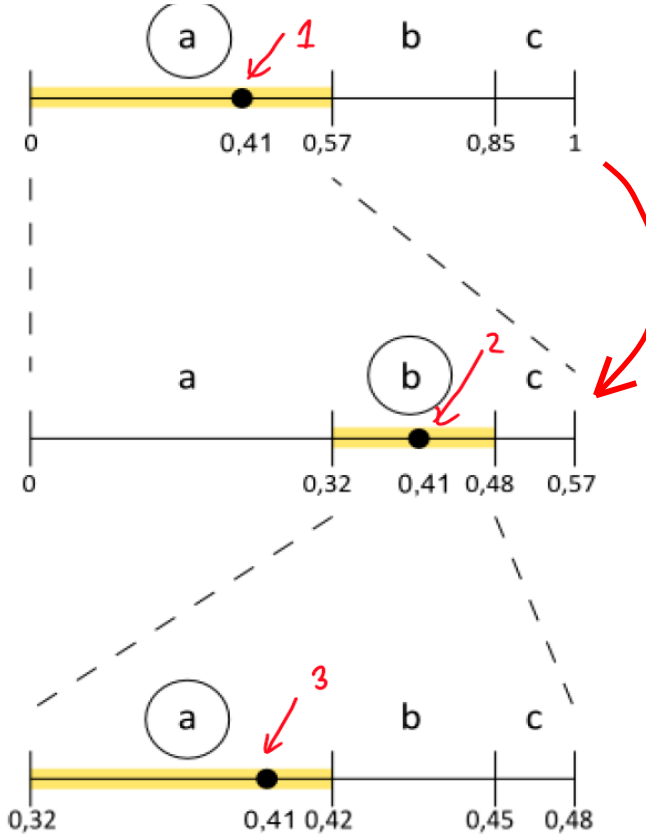
Нормируем
<u>отрезок</u>

Декодируем
0.41 в текст
способ 1

Мы знаем
stop = 3 шага

<u>aba</u>

Декодирование способ 1



$0.41 \in [0; 0.57] \longrightarrow$ **a**

Нормируем

отрезок

$0.41 \in [0.32; 0.48] \longrightarrow$ **b**

Декодируем

0.41 в текст

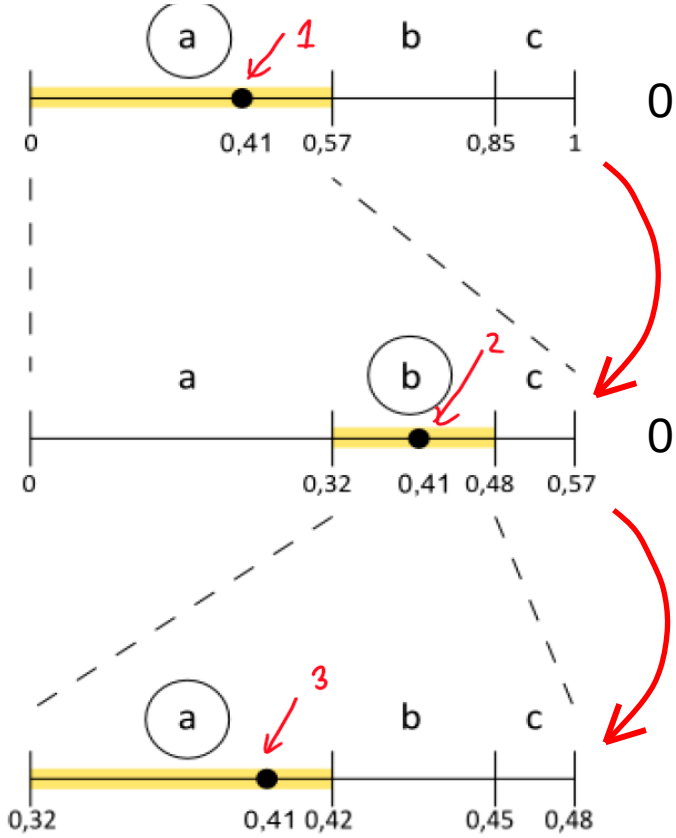
способ 1

Мы знаем

stop = 3 шага

aba

Декодирование способ 1



0.41 ∈ [0; 0.57] ⟶ **a**

Нормируем
отрезок

0.41 ∈ [0.32; 0.48] ⟶ **b**

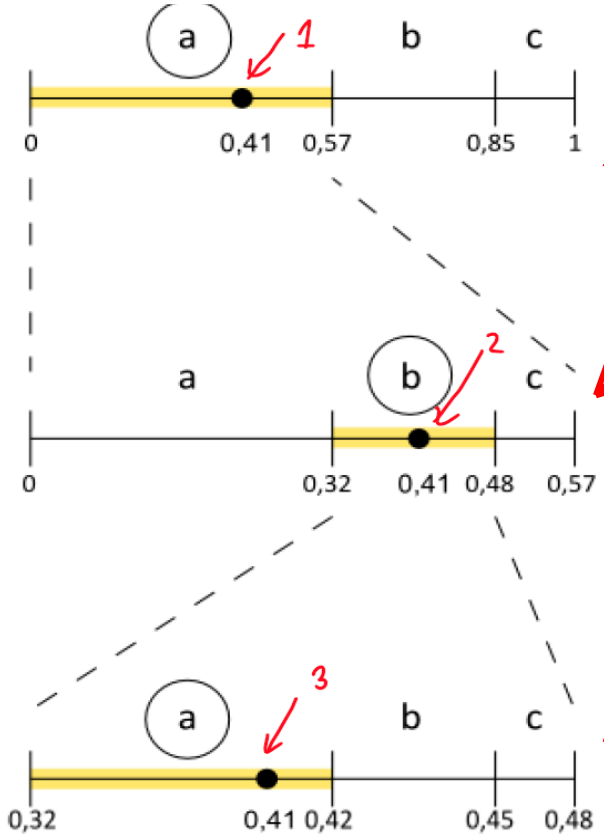
Нормируем
отрезок

Декодируем
0.41 в текст
способ 1

Мы знаем
stop = 3 шага

aba

Декодирование способ 1



0.41 ∈ [0; 0.57] ⟶ **a**

Нормируем
отрезок

0.41 ∈ [0.32; 0.48] ⟶ **b**

Нормируем
отрезок

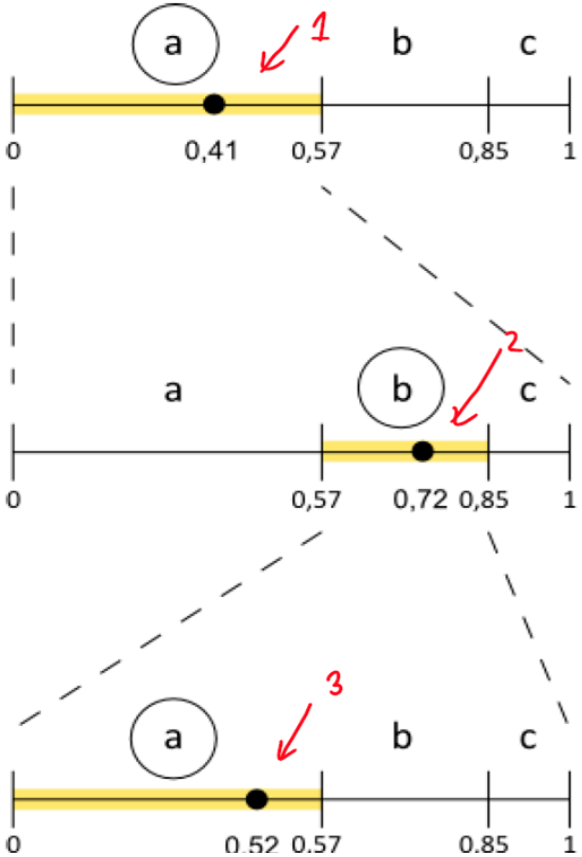
0.41 ∈ [0.32; 0.48] ⟶ **a**

Декодируем
0.41 в текст
способ 1

Мы знаем
stop = 3 шага

aba

Декодирование способ 2



Декодируем

0.41 в текст

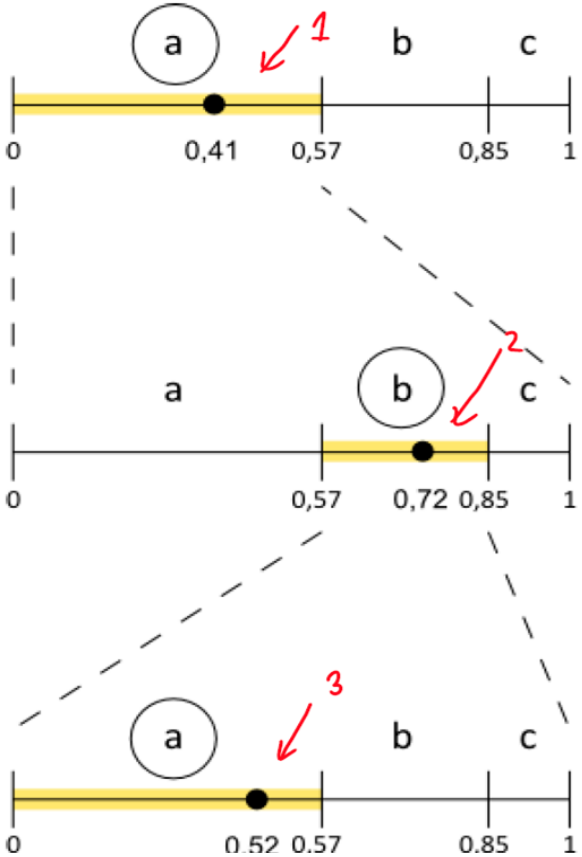
способ 1

Мы знаем

stop = 3 шага

aba

Декодирование способ 2



0.41 ∈ [0; 0.57] ⟶ **a**

Декодируем

0.41 в текст

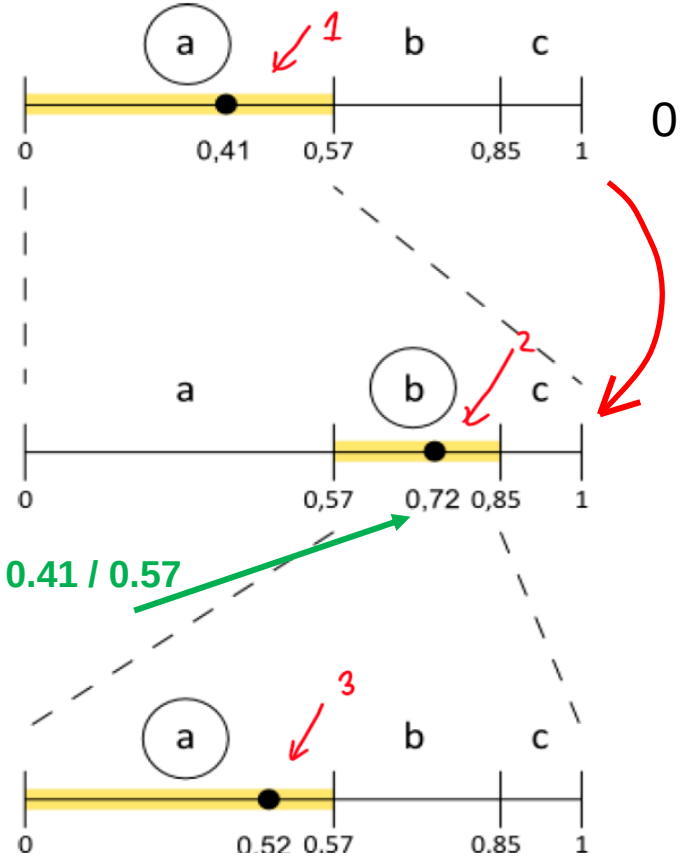
способ 1

Мы знаем

stop = 3 шага

aba

Декодирование способ 2



0.41 ∈ [0; 0.57] ⟶ **a**

Нормируем

число

Декодируем

0.41 в текст

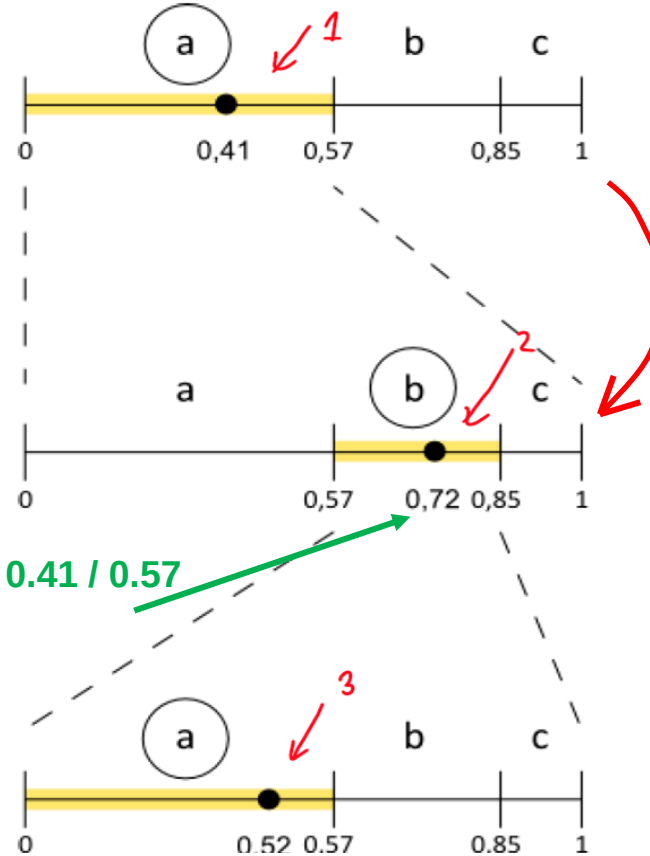
способ 1

Мы знаем

stop = 3 шага

aba

Декодирование способ 2



0.41 ∈ [0; 0.57] ⟶ **a**

Нормируем

<u>число</u>

0.72 ∈ [0.57; 0.85] ⟶ **b**

Декодируем

0.41 в текст

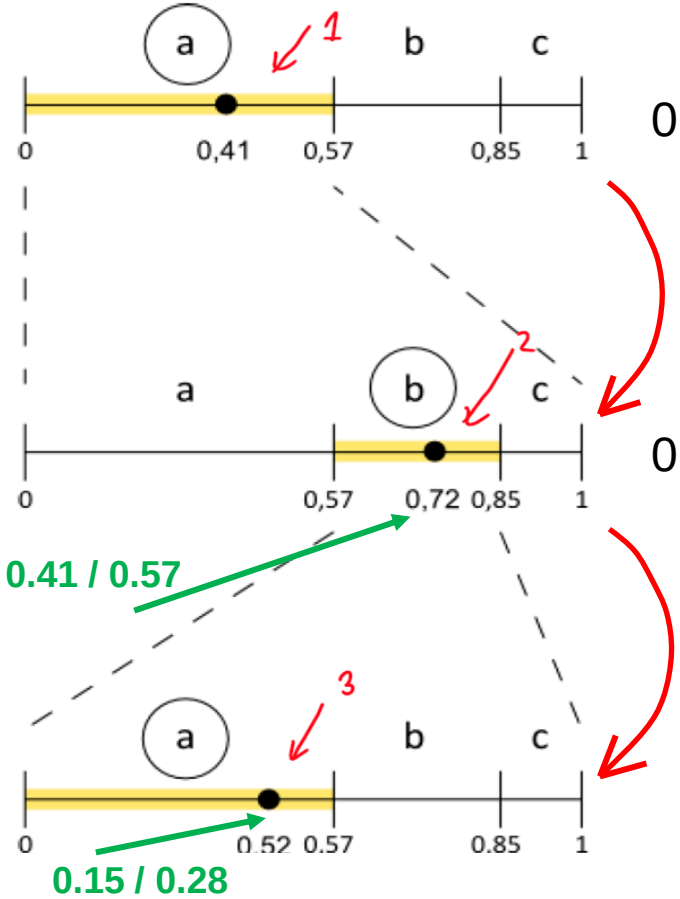
способ 1

Мы знаем

stop = 3 шага

<u>aba</u>

Декодирование способ 2



0.41 ∈ [0; 0.57] ⟶ **a**

Нормируем

<u>число</u>

0.72 ∈ [0.57; 0.85] ⟶ **b**

Нормируем

<u>число</u>

Декодируем

0.41 в текст

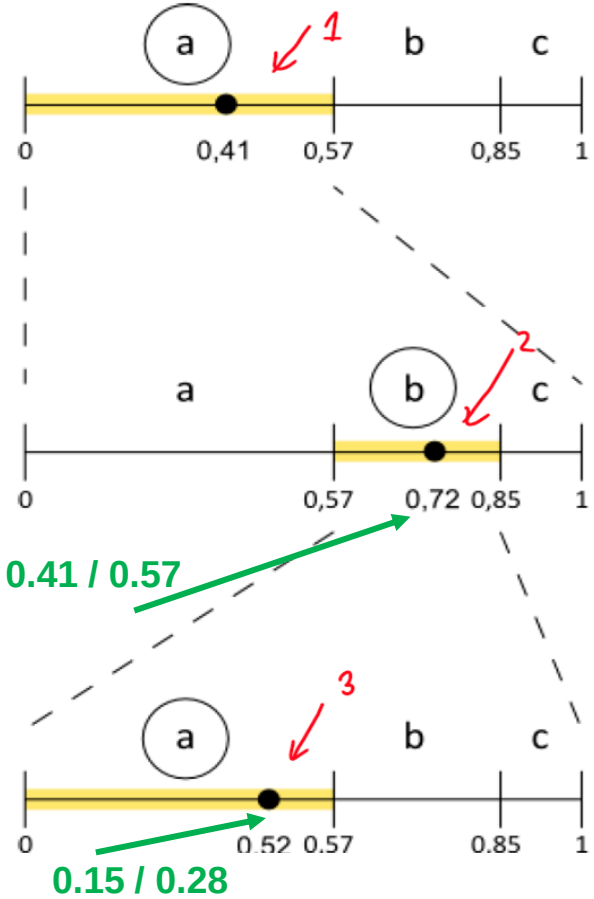
способ 1

Мы знаем

stop = 3 шага

<u>aba</u>

Декодирование способ 2



0.41 ∈ [0; 0.57] ⟶ **a**

Нормируем

<u>число</u>

0.72 ∈ [0.57; 0.85] ⟶ **b**

Нормируем

<u>число</u>

0.52 ∈ [0; 0.57] ⟶ **a**

Декодируем

0.41 в текст

способ 1

Мы знаем

stop = 3 шага

<u>aba</u>

Недостатки:

- Нужен символ конца кодируемого/декодируемого текста ввести его с определенной частотой и по нему определять завершение алгоритма или число символов.

- Знать алфавит и его частоты на стороне декодера.

- Хранение вещественных чисел определенного вида **1/3** могут привести к потерям/искажениям (из за несовершенства представления чисел с плавающей точкой в памяти компьютера).

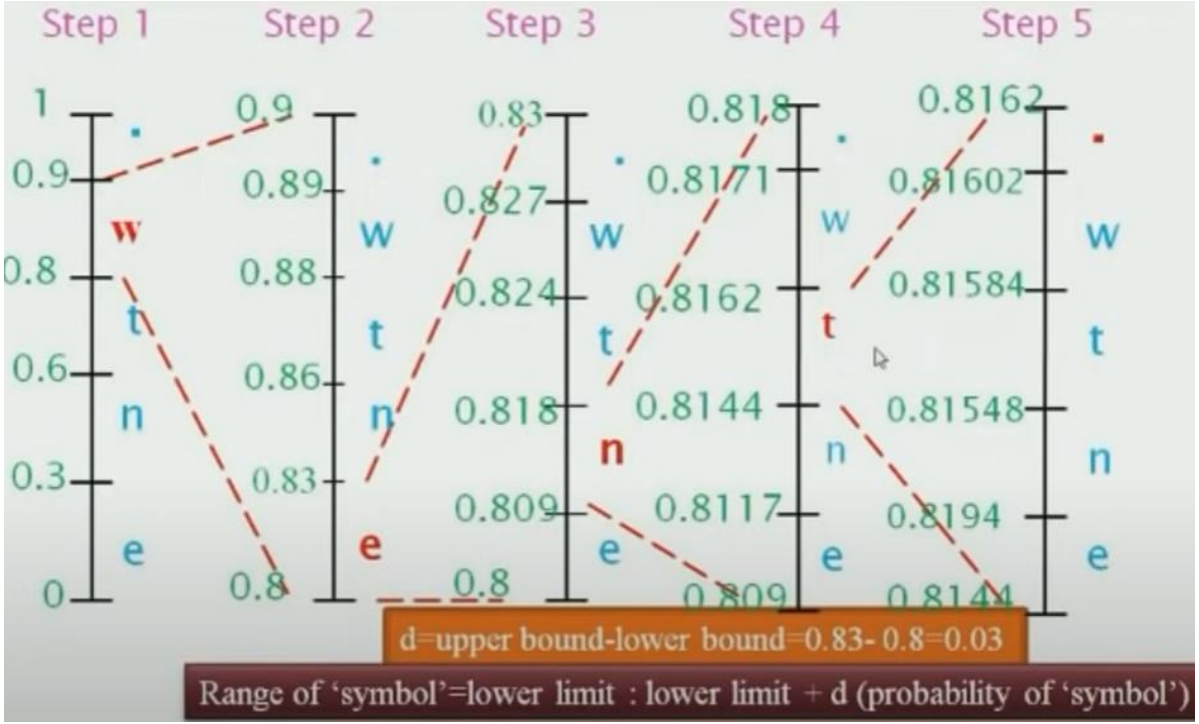
Пример кодирования: went.

e → 0,3
n → 0,3
t → 0,2
w → 0,1
“.” → 0,1

0,81602 < codeword < 0,8162

Generation of Tag:
Tag = (Up Limit of code world +Low limit of code word) / 2

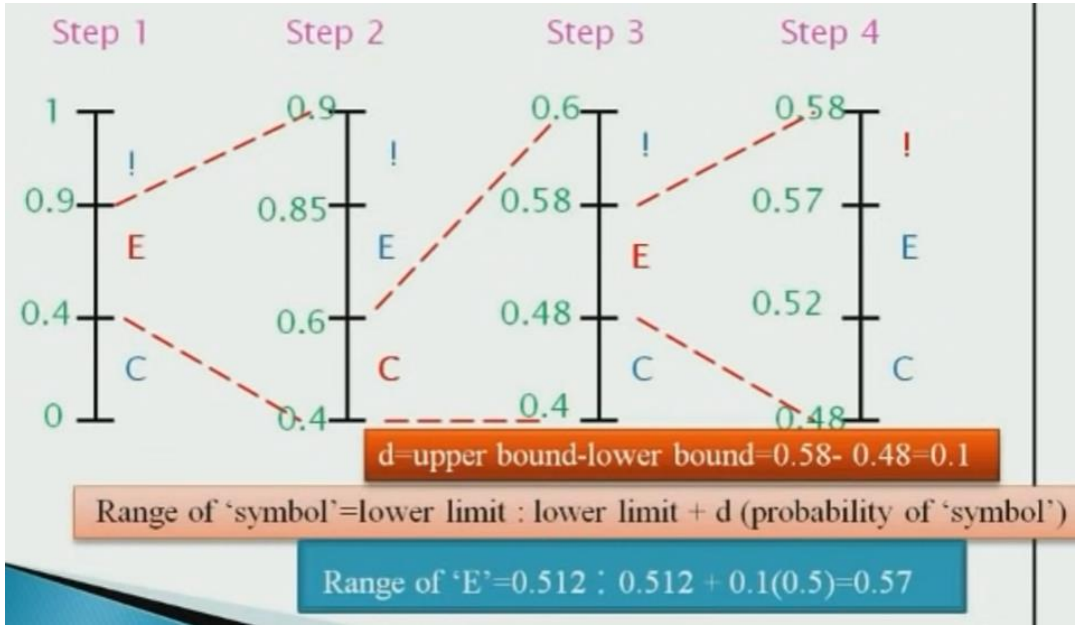
Tag = 0,8162 + 0,81602) / 2
= 0.81611



Декодирование сообщения:

Symbol	!	C	E
Probability	0.1	0.4	0.5

Tag = 0,572



0,57 < codeword < 0,58

=> Ответ: ECE!

```
struct Segment:
  double left
  double right
```

1

1. **Структура сегмента**

```
struct Segment:
  double left
  double right
```

1

```
Segment[m] defineSegments(letters: char[m],
probability: double[m]):
   Segment[m] segment
   double l = 0
   for i = 0 to m - 1
       segment[letters[i]].left = l
       segment[letters[i]].right = l + probability[i]
       l = segment[letters[i]].right
   return segment
```

2

1. **Структура сегмента**
2. **Деление на сегменты**

```
struct Segment:
    double left
    double right
```

1

1. Структура сегмента
2. Деление на сегменты
3. Само кодирование

```
Segment[m] defineSegments(letters: char[m],
probability: double[m]):
    Segment[m] segment
    double l = 0
    for i = 0 to m - 1
        segment[letters[i]].left = l
        segment[letters[i]].right = l + probability[i]
        l = segment[letters[i]].right
    return segment
```

2

```
double arithmeticCoding(letters: char[m], probability: double[m], s: char[n]):
    Segment[m] segment = defineSegments(letters, probability)
    double left = 0
    double right = 1
    for i = 0 to n - 1
        char symb = s[i]
        double newRight = left + (right - left) * segment[symb].right
        double newLeft = left + (right - left) * segment[symb].left
        left = newLeft
        right = newRight
    return (left + right) / 2
```

3

Декодирование

```
struct Segment:
  double left
  double right
  char character
```

1

1. **Структура сегмента**

Декодирование

```
struct Segment:
  double left
  double right
  char character
```

1

1. **Структура сегмента**
2. **Деление на сегменты**

```
Segment[m] defineSegments(letters: char[n],
probability: double[n]):
    Segment[m] segment
    double l = 0
    for i = 0 to m - 1
        segment[i].left = l
        segment[i].right = l + probability[i]
        segment[i].character = letters[i]
        l = segment[i].right
    return segment
```

2

Декодирование

```
struct Segment:
  double left
  double right
  char character
```

1

```
Segment[m] defineSegments(letters: char[n],
probability: double[n]):
   Segment[m] segment
   double l = 0
   for i = 0 to m - 1
       segment[i].left = l
       segment[i].right = l + probability[i]
       segment[i].character = letters[i]
       l = segment[i].right
   return segment
```

2

1. **Структура сегмента**
2. **Деление на сегменты**
3. **Декодирование**

```
string arithmeticDecoding(letters: char[m], probability: double[m],
code: double, n: int):
  Segment[m] segment = defineSegments(letters, probability)
  string s = ""
  for i = 0 to n - 1
    for j = 0 to m - 1
      if code ⩾ segment[j].left and code < segment[j].right
        s += segment[j].character
        code = (code - segment[j].left) / (segment[j].right - segment[j].left)
        break
return s
```

3

Что такое архиваторы?

Для сжатия информации и получения ее самой из сжатого вида используются архиваторы.

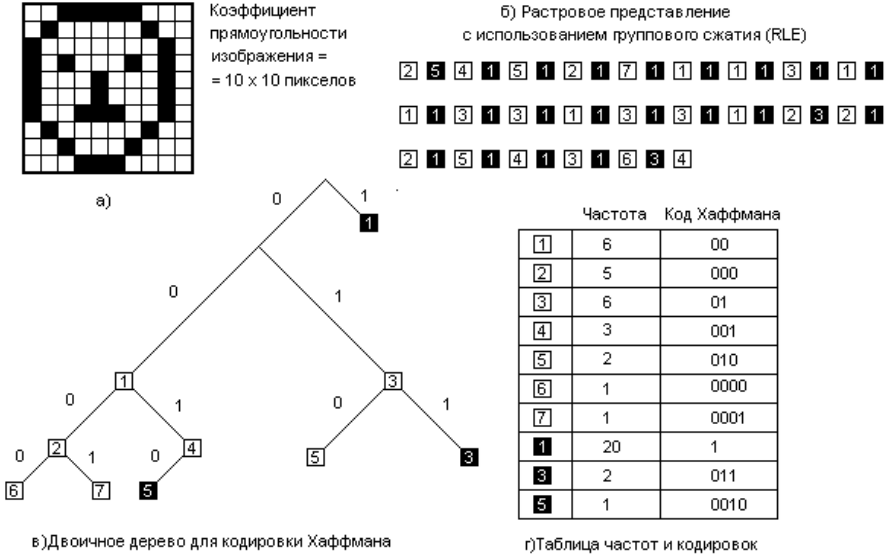
Архиваторы – это программы для создания архивов.

Архивы предназначены для сжатия (упаковки) и хранения данных в удобном компактном виде. В качестве данных обычно выступают файлы и папки.

Чем меньше архив, тем меньше места требуется для его хранения, следовательно, на передачу затрачивается меньшее время. Преимущества архивов очевидны, если учесть, что данные уменьшаются в размере и в 2 раза, и в 5 раз.

Методы сжатия архиваторов. Хаффман

Одним из алгоритмов сжатия является метод Хаффмана, или, как его еще называют — энтропийное кодирование. Основой энтропийного кодирования является сопоставление чаще встречаемому более короткий код. Энтропийное кодирование часто применяется как дополнительный метод, для повышения степени сжатия.



	Частота	Код Хаффмана
1	6	00
2	5	000
3	6	01
4	3	001
5	2	010
6	1	0000
7	1	0001
1	20	1
3	2	011
5	1	0010

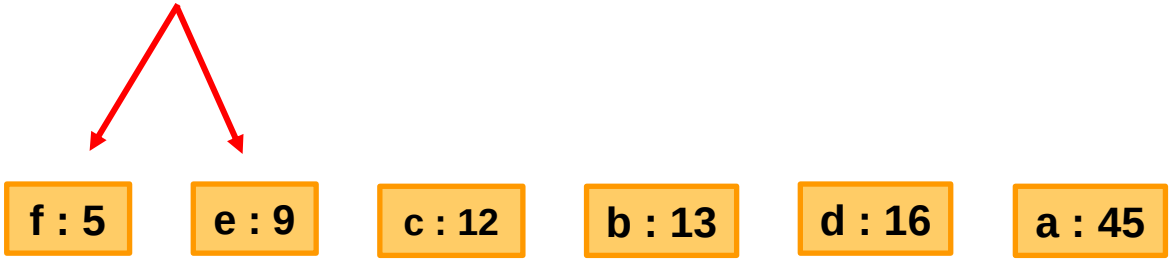
Рис.2.7.Кодировка Хаффмана.

Алгоритм Хаффмана

1. Символы входного алфавита образуют список свободных узлов. Каждый лист имеет вес равный количеству вхождений символа в сжимаемое сообщение.

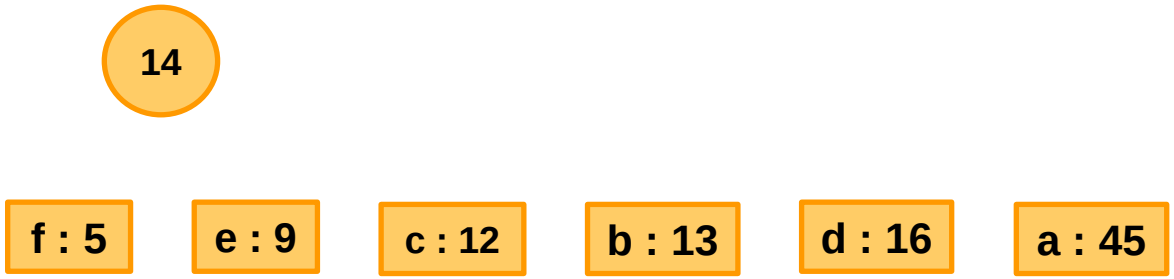
Алгоритм Хаффмана

1. Символы входного алфавита образуют список свободных узлов. Каждый лист имеет вес равный количеству вхождений символа в сжимаемое сообщение.
2. Выбираются два свободных узла дерева с наименьшими весами.



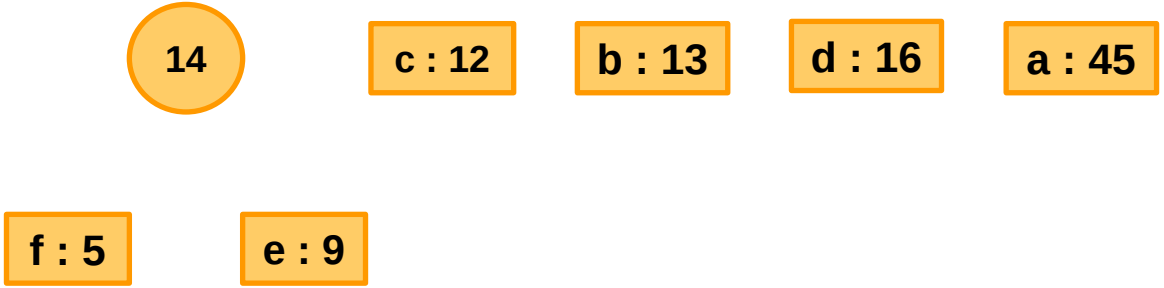
Алгоритм Хаффмана

1. Символы входного алфавита образуют список свободных узлов. Каждый лист имеет вес равный количеству вхождений символа в сжимаемое сообщение.
2. Выбираются два свободных узла дерева с наименьшими весами.
3. Создается их родитель с весом, равным их суммарному весу.



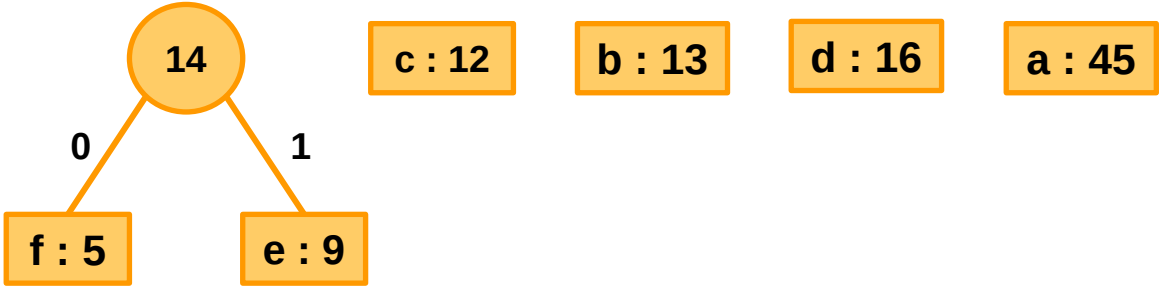
Алгоритм Хаффмана

1. Символы входного алфавита образуют список свободных узлов. Каждый лист имеет вес равный количеству вхождений символа в сжимаемое сообщение.
2. Выбираются два свободных узла дерева с наименьшими весами.
3. Создается их родитель с весом, равным их суммарному весу.
4. Родитель добавляется в список свободных узлов, а два его потомка удаляются из этого списка.



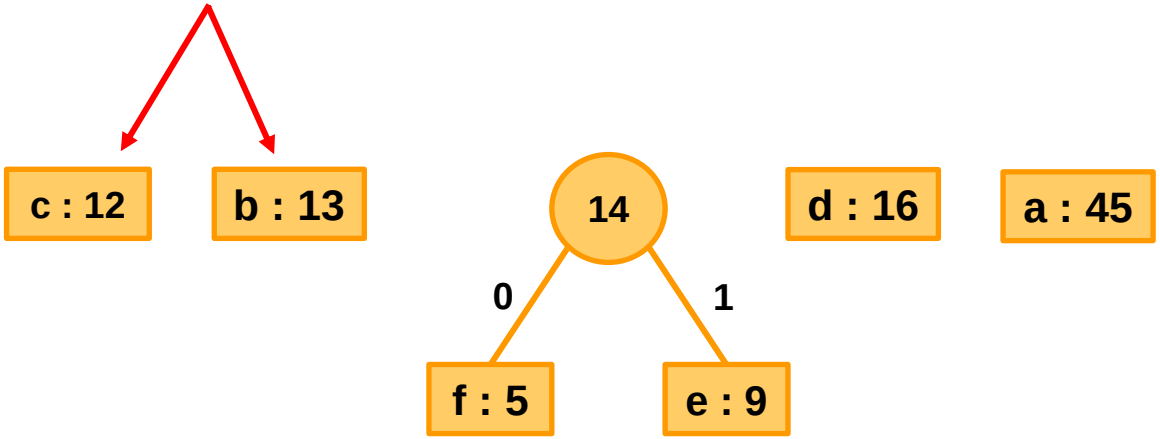
Алгоритм Хаффмана

1. Символы входного алфавита образуют список свободных узлов. Каждый лист имеет вес равный количеству вхождений символа в сжимаемое сообщение.
2. Выбираются два свободных узла дерева с наименьшими весами.
3. Создается их родитель с весом, равным их суммарному весу.
4. Родитель добавляется в список свободных узлов, а два его потомка удаляются из этого списка.
5. Одной дуге, выходящей из родителя, ставится в соответствие бит 1, другой — бит 0.



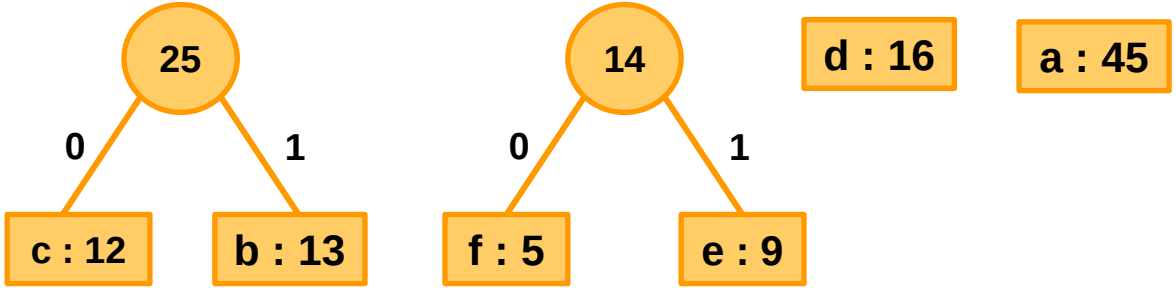
Алгоритм Хаффмана : иллюстрация

6. Шаги, начиная со второго, повторяются до тех пор, пока в списке свободных узлов не останется только один свободный узел.



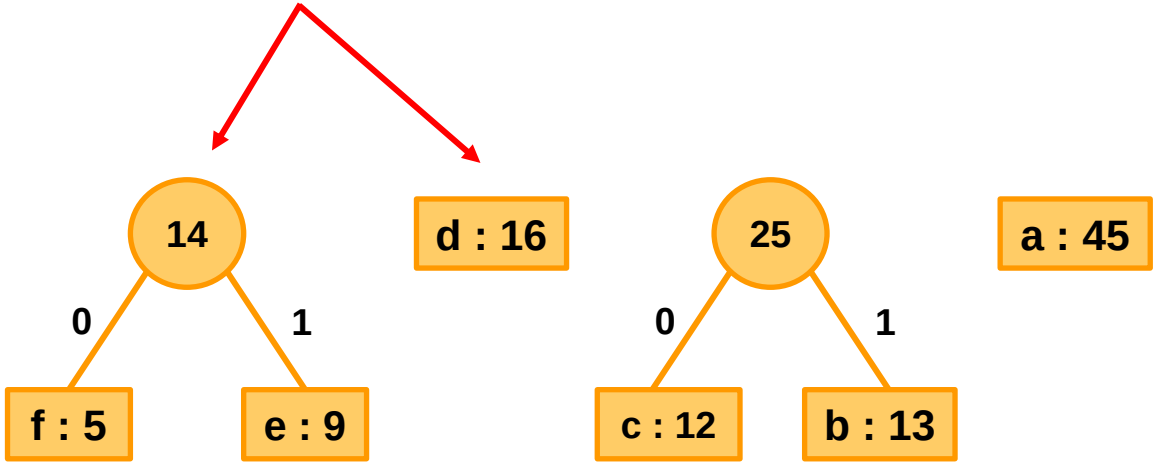
Алгоритм Хаффмана : иллюстрация

6. Шаги, начиная со второго, повторяются до тех пор, пока в списке свободных узлов не останется только один свободный узел.



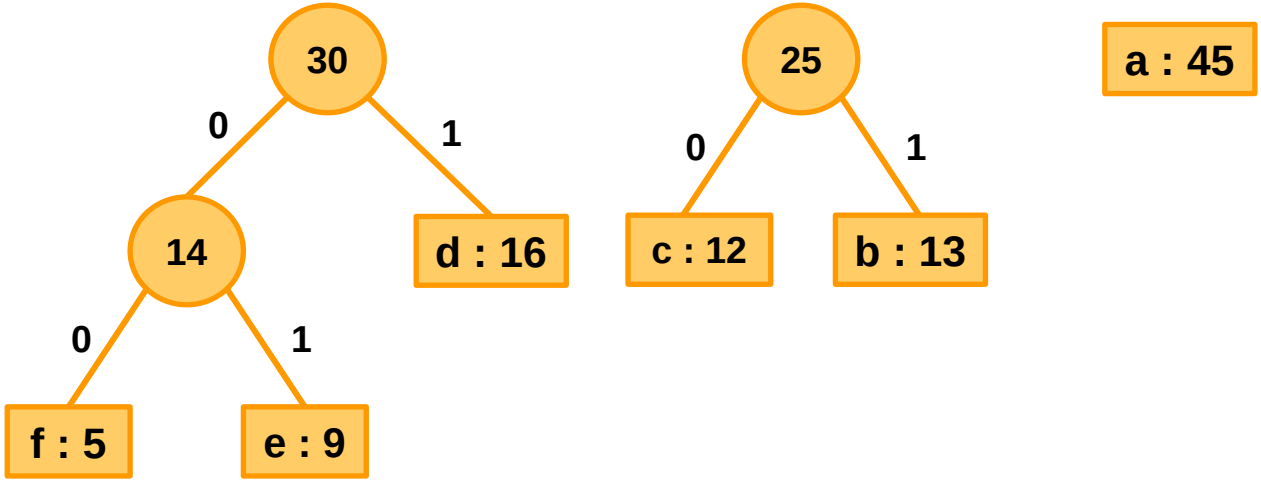
Алгоритм Хаффмана : иллюстрация

6. Шаги, начиная со второго, повторяются до тех пор, пока в списке свободных узлов не останется только один свободный узел.



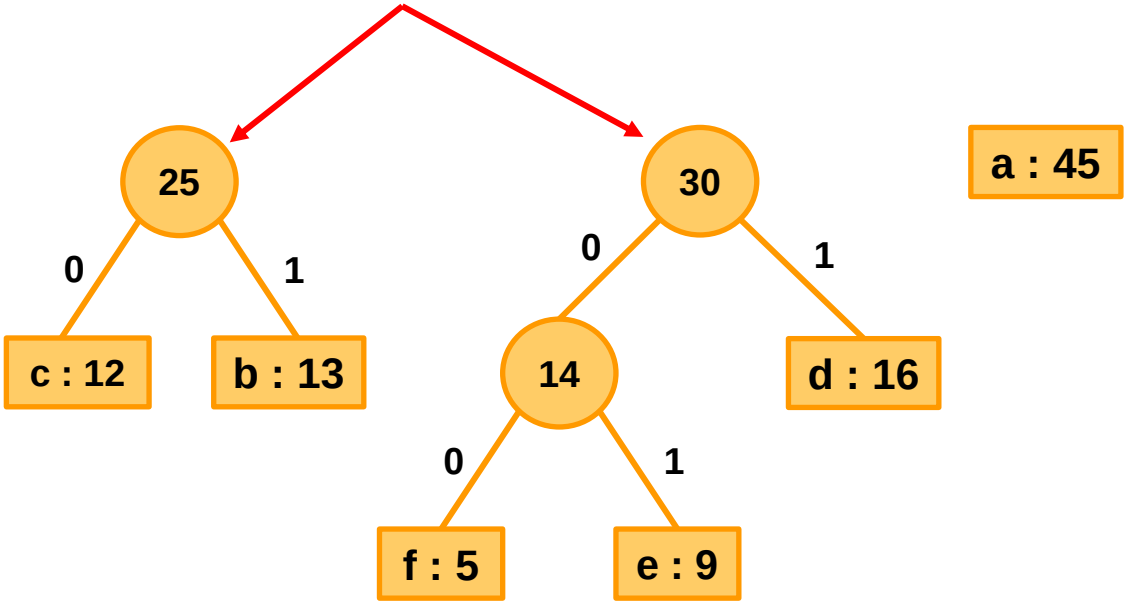
Алгоритм Хаффмана : иллюстрация

6. Шаги, начиная со второго, повторяются до тех пор, пока в списке свободных узлов не останется только один свободный узел.



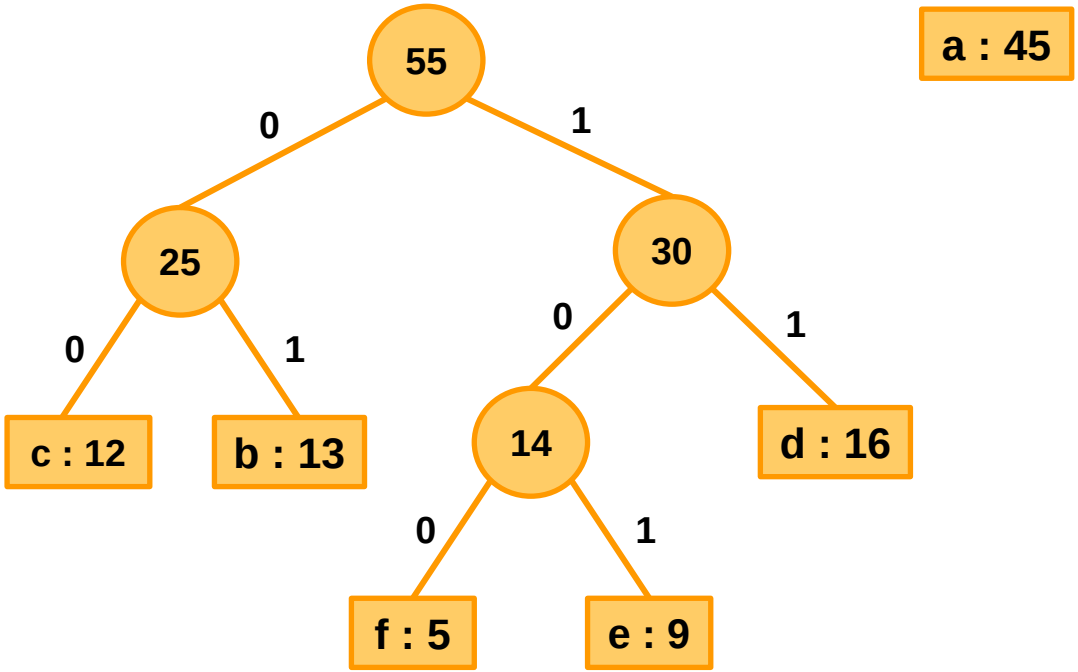
Алгоритм Хаффмана : иллюстрация

6. Шаги, начиная со второго, повторяются до тех пор, пока в списке свободных узлов не останется только один свободный узел.

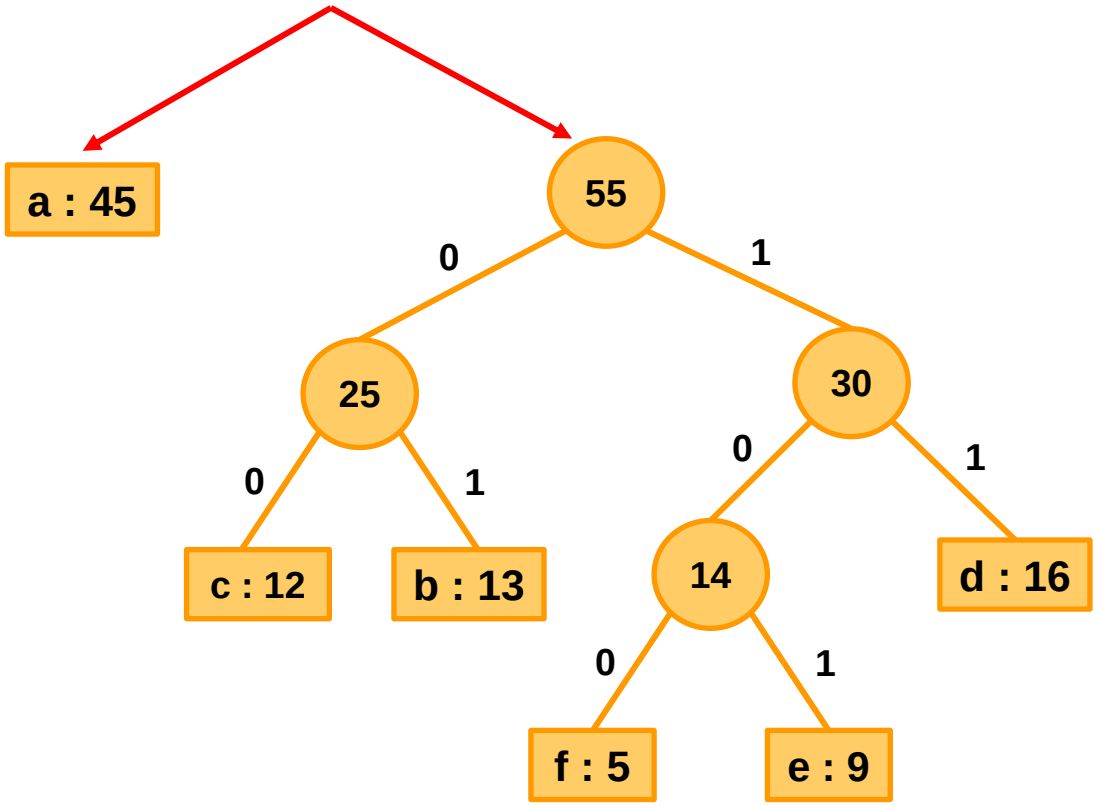


Алгоритм Хаффмана : иллюстрация

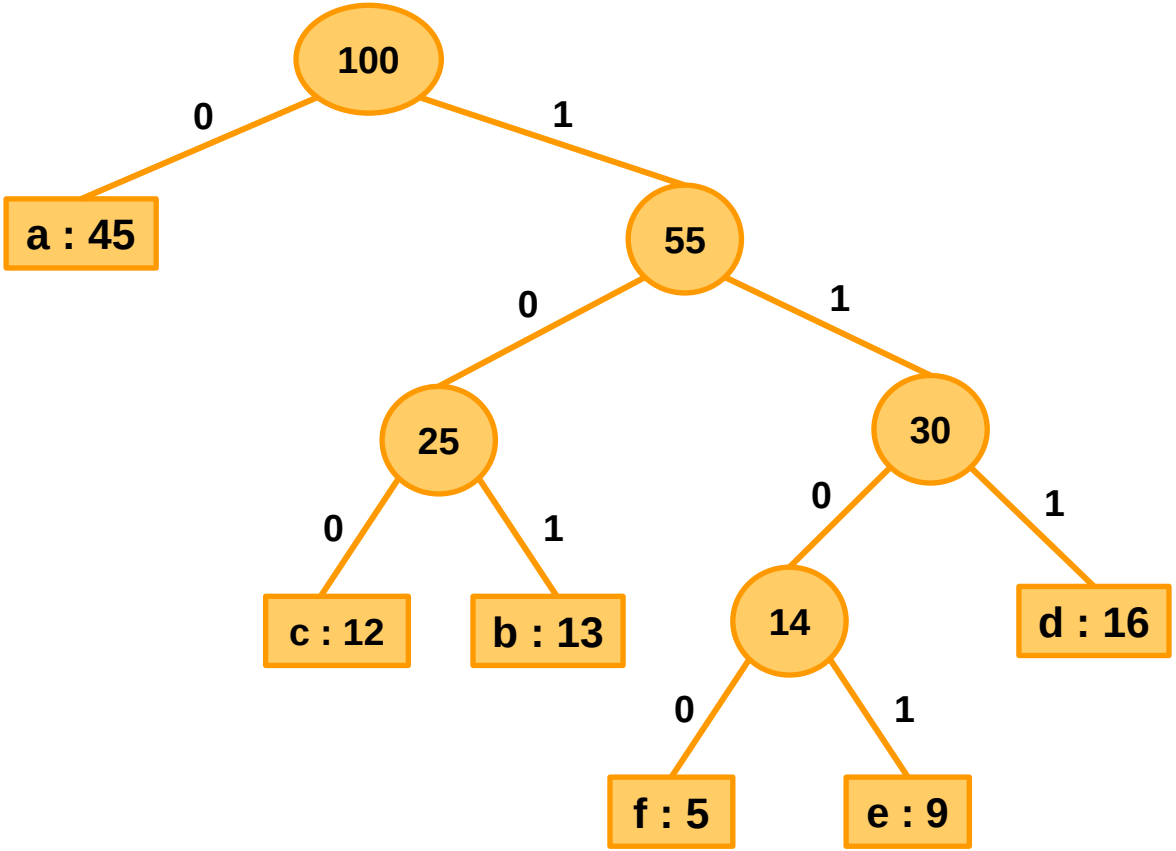
6. Шаги, начиная со второго, повторяются до тех пор, пока в списке свободных узлов не останется только один свободный узел.



Алгоритм Хаффмана : иллюстрация



Алгоритм Хаффмана : иллюстрация

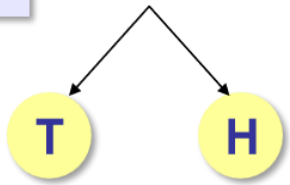


Методы сжатия архиваторов. Хаффман

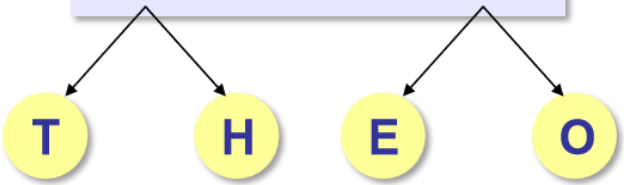
По увеличению частоты:

1) T 50 H 53 E 72 O 89 ␣ 179

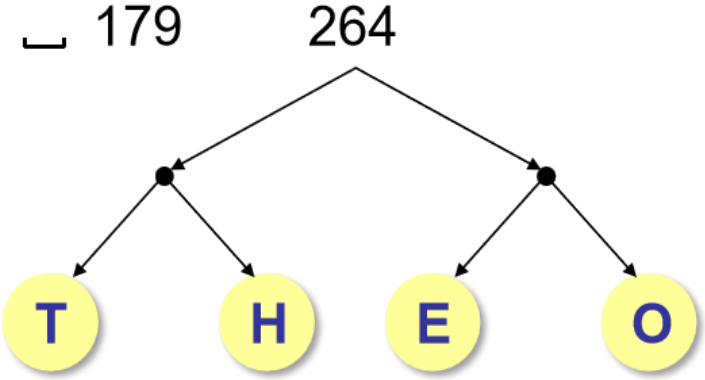
2) E 72 O 89 103 ␣ 179

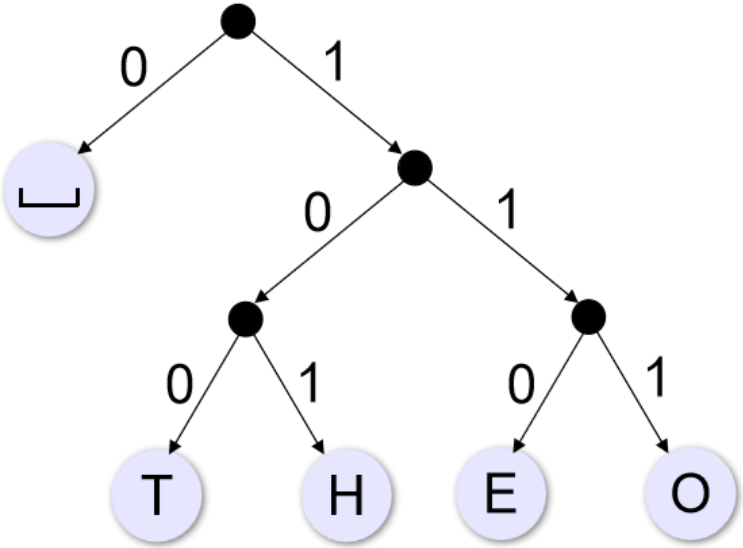


3) 103 161 ␣ 179



Методы сжатия архиваторов. Хаффман





Код Хаффмана:

␣	0	**Т**	100	**Н**	101
		Е	11	**О**	11
			0		1

Построение дерева кодирования

```
HUFFMAN(C)
n = |C|
Q = C
for i = 1 to n − 1
    allocate a new node z
    x = EXTRACT-MIN(Q)
    y = EXTRACT-MIN(Q)
    z.left = x
    z.right = y
    z.freq = x.freq + y.freq
    INSERT(Q, z)
return EXTRACT-MIN(Q)    // the root of the tree is the only node left
```

Используем кучу на минимум, в которой хранятся ноды

Алгоритм Хаффмана

код оптимальный среди алфавитных кодов

- нужно заранее знать частоты символов
- при ошибке в передаче сложно восстановить «хвост»
- не учитывает повторяющиеся последовательности символов

Минимальная избыточность

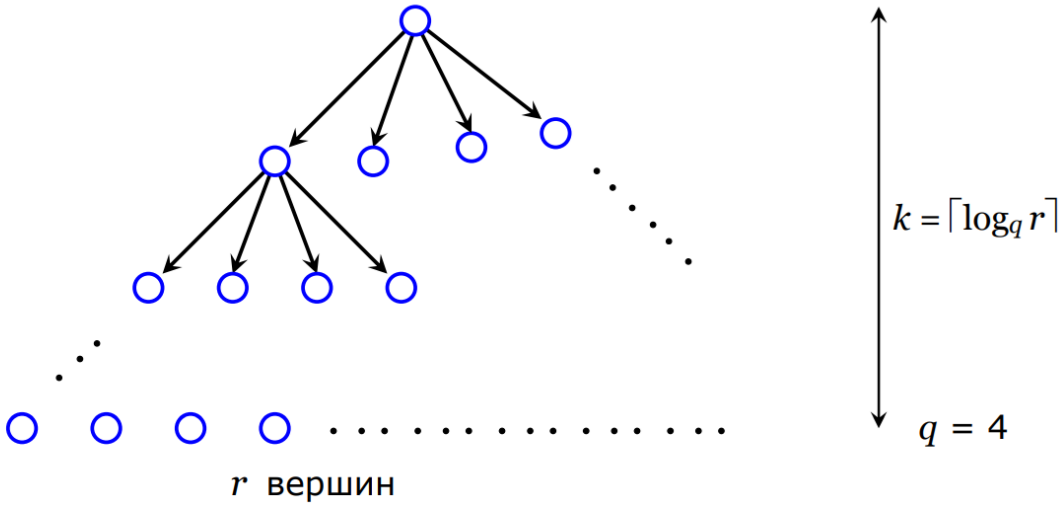
При выборе схемы кодирования естественно учитывать экономичность, т.е. средние затраты времени на передачу и прием сообщений. Предположим, что задан алфавит $\mathbf{A = \{a_1, \ldots, a_r\}}$, $\mathbf{r \geq 2}$, и набор вероятностей $\mathbf{(p_1, \ldots, p_r)}$, $\mathbf{p_1 + \ldots + p_r = 1}$ появлений букв $\mathbf{a_1, \ldots, a_r}$. Тогда избыточностью кодирования схемой Σ называется величина:

$$\mathbf{lcp = lcp(\Sigma) = l_1 * p_1 + \ldots + l_r * p_r,}$$

т.е. математическое ожидание длины элементарного кода, $\mathbf{l_i}$ — длина кодового слова для $\mathbf{a_i}$.

Минимальная избыточность

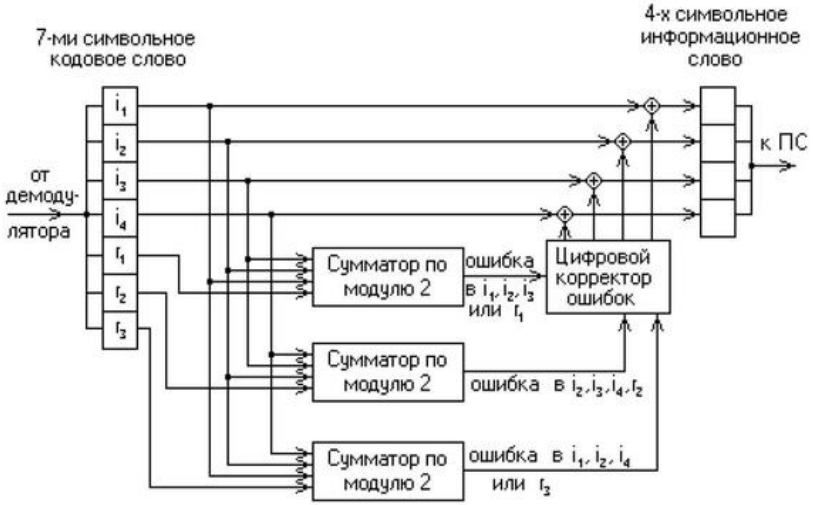
Чем меньше **lср**, тем меньше в среднем избыточность.
Коды, определяемые схемами избыточности, называются кодами с минимальной избыточностью или <u>кодами Хаффмана</u>.



Код Хэмминга

Каналы передачи данных ненадежны (шумы, наводки и т.д.), да и само оборудование обработки информации работает со сбоями. По этой причине важную роль приобретают механизмы детектирования ошибок.

Код Хемминга – это блочный код, позволяющий исправлять одиночные и фиксировать двойные ошибки, разработанный Ричардом Хеммингом в сороковых годах прошлого столетия.



Код Хэмминга

Идея кодов Хемминга заключается в разбиении данных на блоки фиксированной длины и вводе в эти блоки контрольных бит, дополняющих до четности несколько пересекающихся групп, охватывающих все биты блока.

Ричард Хемминг рассчитал минимальное количество проверочных бит, позволяющих однозначно исправлять однократные ошибки.

Код Хемминга

Проверочная матрица задает номера бит кодовой комбинации, которые участвуют в каждой из **r** проверок. В качестве проверочных бит удобно выбрать такие, которые входят только один раз в каждую проверку, т. е. те, для которых $\mathbf{q_i}$ содержит лишь один ненулевой бит.

$$H = \begin{array}{c} \begin{array}{ccccccc} q_1 & q_2 & q_3 & q_4 & \cdots & q_{n-1} & q_n \end{array} \\ \begin{pmatrix} 1 & 0 & 1 & 0 & \dots & 1 & 1 \\ 0 & 1 & 1 & 0 & \dots & 1 & 1 \\ 0 & 0 & 0 & 1 & \dots & 1 & 1 \\ 0 & 0 & 0 & 0 & \dots & 1 & 1 \\ \dots & \dots & \dots & \dots & \dots & 1 & 1 \\ 0 & 0 & 0 & 0 & \dots & 0 & 1 \end{pmatrix} \end{array}.$$

Код Хемминга

Для того, чтобы понять, за какие биты отвечает каждых контрольный бит необходимо понять очень простую закономерность: контрольный бит с номером **N** контролирует все последующие **N** бит через каждые **N** бит, начиная с позиции **N**.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	
0	0	0	0	1	0	0	0	0	1	0	0	0	0	1	0	1	1	1	0	1	
X		X		X		X		X		X		X		X		X		X		X	1
	X	X			X	X			X	X			X	X			X	X			2
			X	X	X	X					X	X	X	X					X	X	4
							X	X	X	X	X	X	X	X							8
															X	X	X	X	X	X	16

Здесь знаком **«X»** обозначены те биты, которые контролирует контрольный бит, номер которого справа. То есть, к примеру, бит номер **12** контролируется битами с номерами **4** и **8**. Ясно, что чтобы узнать какими битами контролируется бит с номером **N** надо просто разложить **N** по степеням двойки.

Код Хемминга

Символ	ACSII код	Бинарное представление
h	68	01000100
a	61	00111101
b	62	00111110
r	72	01001000

Код Хэмминга состоит из двух частей.

- Первая часть кодирует исходное сообщение, вставляя в него в определённых местах контрольные биты (вычисленные особым образом).

- Вторая часть получает входящее сообщение и заново вычисляет контрольные биты (по тому же алгоритму, что и первая часть).

Если все вновь вычисленные контрольные биты совпадают с полученными, то сообщение получено без ошибок.
В противном случае, выводится сообщение об ошибке и при возможности ошибка исправляется.

Код Хемминга

h	a
01000100	00111101

b	r
00111110	01001000

На этом этапе стоит определиться с, так называемой, длиной информационного слова, то есть длиной строки из нулей и единиц, которые мы будем кодировать.

- Допустим, у нас длина слова будет равна **16**.
- Таким образом, нам необходимо разделить наше исходное сообщение **(«habr»)** на блоки по **16 бит**, которые мы будем потом кодировать отдельно друг от друга.
- Так как один символ занимает в памяти **8 бит**, то в одно кодируемое слово помещается ровно **два** ASCII символа. Итак, мы получили две бинарные строки по **16 бит**

Код Хемминга

h	a
01000100	00111101

После этого процесс кодирования распараллеливается, и две части сообщения (**«ha»** и **«br»**) кодируются независимо друг от друга. Рассмотрим, как это делается на примере первой части.

Прежде всего, необходимо вставить контрольные биты.

Они вставляются в строго определённых местах — это позиции с номерами, равными степеням двойки. В нашем случае (при длине информационного слова в **16 бит**) это будут позиции **1, 2, 4, 8, 16**. Соответственно, у нас получилось **5** контрольных бит (выделены красным цветом).

h	a
000010000100	001011101

Таким образом, длина всего сообщения увеличилась на **5 бит**. До вычисления самих контрольных бит, мы присвоили им значение **«0»**.

Код Хемминга

Здесь **4 бита** не уместилось, поэтому записываем сколько можем => **2 бита**

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	
0	0	0	0	1	0	0	0	0	1	0	0	0	0	1	0	1	1	1	0	1	
X		X		X		X		X		X		X		X		X		X		X	1
	X	X			X	X			X	X			X	X			X	X			2
			X	X	X	X					X	X	X	X					X	X	4
							X	X	X	X	X	X	X	X							8
															X	X	X	X	X	X	16

Например, контрольный бит с номером **4** контролирует группы из **4 бит**, начиная с позиции **4**. Пропуск между группами также имеет размер **4 бита.**

Код Хемминга

Но как же вычислить значение каждого контрольного бита?

Делается это очень просто:

- берём каждый контрольный бит и смотрим сколько среди контролируемых им битов единиц,
- получаем некоторое целое число и, если оно чётное, то ставим ноль, в противном случае ставим единицу.

Высчитав контрольные биты для нашего информационного слова получаем следующее:

h	a
100110000100	001011101

Для второй части аналогично:

b	r
100101101110	010101000

Декодирование

Теперь, допустим, мы получили закодированное первой частью алгоритма сообщение, но оно пришло к нас с ошибкой.

К примеру мы получили такое (11-ый бит передался неправильно):

h	a
100110000110	001011101

Декодирование

h	a
10011000110	001011101

Вся вторая часть алгоритма заключается в том, что необходимо заново вычислить все контрольные биты (так же как и в первой части) и сравнить их с контрольными битами, которые мы получили. Так, посчитав контрольные биты с неправильным **11-ым** битом мы получим такую картину:

h	a
01011001011 0	001011101

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21

Как мы видим, контрольные биты под номерами: **1, 2, 8** не совпадают с такими же контрольными битами, которые мы получили.

Теперь просто сложив номера позиций неправильных контрольных бит **(1 + 2 + 8 = 11)** мы получаем позицию ошибочного бита. Теперь просто инвертировав его и отбросив контрольные биты, мы получим исходное сообщение в первозданном виде! Абсолютно аналогично поступаем со второй частью сообщения.

Реализация

Зададим начальную последовательность

```
$input = "1001000"; // задаем последовательость
$m = strlen($input); // узнаем длину
```

Для начала узнаем количество контрольных разрядов:

```
$k = 2; // задаем самое минимально возможное k
$l2n = log10(2); //
while ($k * $l2n < log10($k + $m + 1)) $k++; // вычисляем по формуле
минимальное k
```

Реализация

Запишем информационные разряды на свои места

```
  $result = Array(); // заводим массив разрядов

 for ($i = 1, $j=0, $n = $k+$m; $i <= $n; $i++) { // проходим по всем разрядам
   if (($i & $i-1) == 0) { // если степень двойки, то есть контрольный разряд,
то пропускаем
     $result[$i] = '-';
     continue;
   }
   $result[$i] = $input[$j++]; // в информационные разряды записываем исходные
данные
 }
```

Реализация

Узнаем контрольную сумму:

```
$control = 0;
 for ($i = 1, $n = $k+$m; $i <= $k+$m; $i++) { // проходим по всем разрядам
   if ($result[$i] == 1)
     $control ^= $i; // если единица, то суммируем по модулю два
 }
 $control = decbin($control); // преобразовываем в двоичный код
 for ($i=0, $n = strlen($control); $i<$k-$n; $i++) // дописываем спереди нули
   $control = "0".$control;
```

И записываем контрольную сумму в контрольные разряды:

```
for ($i = 0, $j = 1, $n = strlen($control); $i < $n; $i++, $j*=2) {
   $result[$j] = $control[$i];
 }
```

Реализация

Выдаем результат:

```
$output = "";
 for ($i= 1, $n = count($result); $i <= $n; $i++)
   $output .= $result[$i];
 echo $output;
```

Декодирование

```
for (i = 0; i < len; i++)
        if (input[i] != '0' && input[i] != '1')
       {
           printf("Invalid code.\n");
           return -1;
       }

 mistake = find_mistake();
     if (mistake)
         {
         input[mistake - 1] ^= '0' ^ '1';
         mistake = find_mistake();

         if (mistake)
            {
             printf("Invalid code.\n");
             return -1;
            }
         }
```

```
printf("Fixed code: %s\n", input);
for (i = 0; i < len; i++)
     if (i + 1 == b)
          b <<= 1;
     else
        info[info_pos++] = input[i];
        info[info_pos] = '\0';
        printf("Informational string: %s\n", info);
        return 0;
```

Методы сжатия архиваторов. LZ

Словарный метод (LZ)

Наиболее распространенный метод. Используется словарь, состоящий из последовательностей данных или слов. При сжатии эти слова заменяются на их коды из словаря. В наиболее распространенном варианте реализации в качестве словаря выступает сам исходный блок данных.

ВХОДНАЯ ФРАЗА (В СЛОВАРЬ)	КОД	ПОЗИЦИЯ СЛОВАРЯ
		0
"К"	<0,'К'>	1
"Р"	<0,'Р'>	2
"А"	<0,'А'>	3
"С"	<0,'С'>	4
"Н"	<0,'Н'>	5
"АЯ"	<3,'Я'>	6
	<0,' '>	7
"КР"	<1,'Р'>	8
"АС"	<3,'С'>	9
"КА"	<1,'А'>	10

Методы сжатия архиваторов. LZ

Основным параметром словарного метода является размер словаря. Чем больше словарь, тем больше эффективность. Однако для неоднородных данных чрезмерно большой размер может быть вреден, так как при резком изменении типа данных словарь будет заполнен неактуальными словами.

Кодируемая последовательность	Код	Последовательность в словарь	Код
К	00001	КУ	01011
У	00010	УК	01100
КУ	01011	КУШ	01101
Ш	00011	ШК	01110
К	00001	КА	01111
А	00100	АК	10000
КУ	01011	КУК	10001
КУШ	01101	КУШО	10010
О	00101	ОН	10011
Н	00110	НК	10100
КУК	10001	КУКУ	10101
У	00010	УП	10110
П	00111	ПИ	10111
И	01000	ИЛ	11000
Л	01001	ЛА	11001
АК	10000	АКА	11010
А	00100	АП	11011
П	00111	ПЮ	11100
Ю	01010	ЮШ	11101
Ш	00011	ШО	11110
ОН	10011		

Методы сжатия архиваторов. LZ

Для эффективной работы данного метода при сжатии требуется дополнительная память. Приблизительно на порядок больше, чем нужно для исходных данных словаря. Существенным преимуществом словарного метода является простая и быстрая процедура распаковки. Дополнительная память при этом не требуется. Такая особенность особенно важна, если необходим оперативный доступ к данным.

ВХОДНОЙ КОД	ПЕЧАТЬ	СЛОВАРЬ	ПОЗИЦИЯ СЛОВАРЯ
		ASCII+	0-255
0’К’	"К"	"КР"	256
0’Р’	"Р"	"РА"	257
0’А’	"А"	"АС"	258
0’С’	"С"	"СН"	259
0’Н’	"Н"	"НА"	260
0’А’	"А"	"АЯ"	261
0’Я’	"Я"	"Я "	262
0’ ’		"К"	263
<256>	"КР"	"КРА"	264
<258>	"АС"	"АСК"	265
0’К’	"К"	"КА"	266
0’А’	"А"		

Методы сжатия архиваторов. LZ

Идеи:

- кодировать не отдельные символы, а блоки
- последовательностям символов присваиваются числовые коды
- новая цепочка ⇒ занесение в словарь с новым кодом

- словарь строится по мере получения данных
- не нужны частоты символов ⇒ за один проход!

Методы сжатия архиваторов. LZ

Кодирование

- Начало.
- **Шаг 1.** Все возможные символы заносятся в словарь. Во входную фразу X заносится первый символ сообщения.
- **Шаг 2.** Считать очередной символ Y из сообщения.
- **Шаг 3.** Если Y — это символ конца сообщения, то выдать код для X, иначе:
  - Если фраза XY уже имеется в словаре, то присвоить входной фразе значение XY и перейти к **Шагу 2** ,
  - Иначе выдать код для входной фразы X, добавить XY в словарь и присвоить входной фразе значение Y. Перейти к **Шагу 2.**
- Конец.

Методы сжатия архиваторов. LZ

Декодирование

- Начало.
- **Шаг 1.** Все возможные символы заносятся в словарь. Во входную фразу X заносится первый код декодируемого сообщения.
- **Шаг 2.** Считать очередной код Y из сообщения.
- **Шаг 3.** Если Y — это конец сообщения, то выдать символ, соответствующий коду X, иначе:
  - Если фразы под кодом XY нет в словаре, вывести фразу, соответствующую коду X, а фразу с кодом XY занести в словарь.
  - Иначе присвоить входной фразе код XY и перейти к **Шагу 2** .
- Конец.

LZ78

Изначально словарь пуст, а алгоритм пытается закодировать первый символ.
На каждой итерации мы пытаемся увеличить кодируемый префикс, пока такой префикс есть в словаре.
Кодовые слова такого алгоритма будут состоять из двух частей — номера в словаре самого длинного найденного префикса **(pos)** и символа, который идет за этим префиксом **(next)**. При этом после кодирования такой пары префикс с приписанным символом добавляется в словарь, а алгоритм продолжает кодирование со следующего символа.

LZ78

Пример

В этот раз для примера возьмем строку $abacababacabc$.

Словарь	Осталось обработать	Найденный префикс	Код	Примечание
$\varnothing$	$abacababacabc$	—	$\langle 0, a\rangle$	В словаре ничего не нашлось, вместо номера в словаре указываем 0
a	$bacababacabc$	—	$\langle 0, b\rangle$	
a, b	$acababacabc$	a	$\langle 1, c\rangle$	Нашелся префикс a (слова в словаре нумеруются с 1), добавляем ac
a, b, ac	$ababacabc$	a	$\langle 1, b\rangle$	
a, b, ac, ab	$abacabc$	ab	$\langle 4, a\rangle$	
a, b, ac, ab, aba	$cabc$	—	$\langle 0, c\rangle$	
a, b, ac, ab, aba, c	abc	ab	$\langle 4, c\rangle$	

Результатом кодирования является список пар: $[\langle 0, a\rangle, \langle 0, b\rangle, \langle 1, c\rangle, \langle 1, b\rangle, \langle 4, a\rangle, \langle 0, c\rangle, \langle 4, c\rangle]$

LZ78

```
struct Node:
    int pos   // номер слова в словаре
    char next
```

```
list<Node> encodeLZ78(string s):
    string buffer = ""                              // текущий префикс
    map<string, int> dict = {}                      // словарь
    list<Node> ans = []                             // ответ
    for i = 0 .. s.length - 1:
        if dict.hasKey(buffer + s[i]):              // можем ли мы увеличить
префикс
            buffer += s[i]
        else:
            ans.push({dict[buffer], s[i]})          // добавляем пару в ответ
            dict[buffer + s[i]] = dict.length + 1   // добавляем слово в
словарь
            buffer = ""                             // сбрасываем буфер
    if not (buffer is empty): // если буффер не пуст - этот код уже был,
нужно его добавить в конец словаря
        last_ch = buffer.peek() // берем последний символ буффера, как
"новый" символ
        buffer.pop() // удаляем последний символ из буфера
        ans.push({dict[buffer], last_ch}) // добавляем пару в ответ
    return ans
```

LZ78

Декодирование

Декодирование происходит аналогично кодированию, на основе декодируемой информации строим словарь и берем из него значения.

```
string decodeLZ78(list<Node> encoded):
    list<string> dict = [""]                        // словарь, слово с номером 0 — пустая строка
    string ans = ""                                 // ответ
    for node in encoded:
        word = dict[node.pos] + node.next           // составляем слово из уже известного из словаря и новой буквы
        ans += word                                 // приписываем к ответу
        dict.push(word)                             // добавляем в словарь
    return ans
```